

Q1:What is the difference between CachedThreadPool and FixedThreadPool?

A1: A cached thread pool does not have a fixed number of threads that the pool can use, while fixed thread pools have a pre defined max number of threads it can allocate. Idle threads within a cached thread pool will be destroyed after a fixed time of idling, while in fixed thread pools idle threads stay alive. When all FixedThreads are in use, fixed thread pool will queue up the next task, while cached threads simply generate another thread. Cached threading can be dangerous though due to an excess load creating more threads, eventually you can run out of memory triggering an error, fixed threads simply put excess load into a queue to be processed once a thread becomes available.

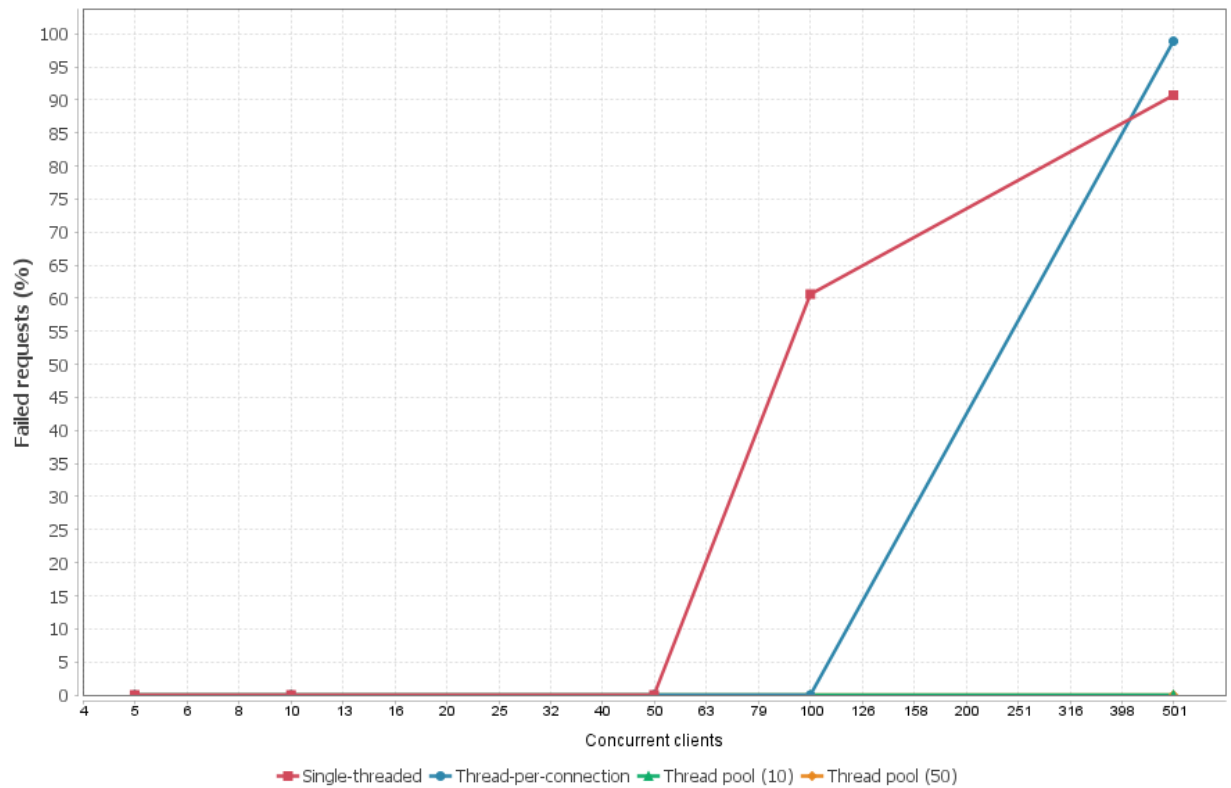
In our example of 500 concurrent clients, the cached pool will grow to roughly 500 threads as long as enough memory is provided to the application, while for a fixedThreadPool holds N number of threads, which in our scenario n can be 10 or 50 depending on the scenario. This means that the thread pool example can only support n clients at a time before the other clients start being added into the queue. The key difference to be stated between the two is fixedThreadPools handle overhead via a queue depth and cachedThreadPools handle overhead with thread depth.

Q2: Is there any Executor Service for Thread Per Connection? If not, use the original implementation and justify

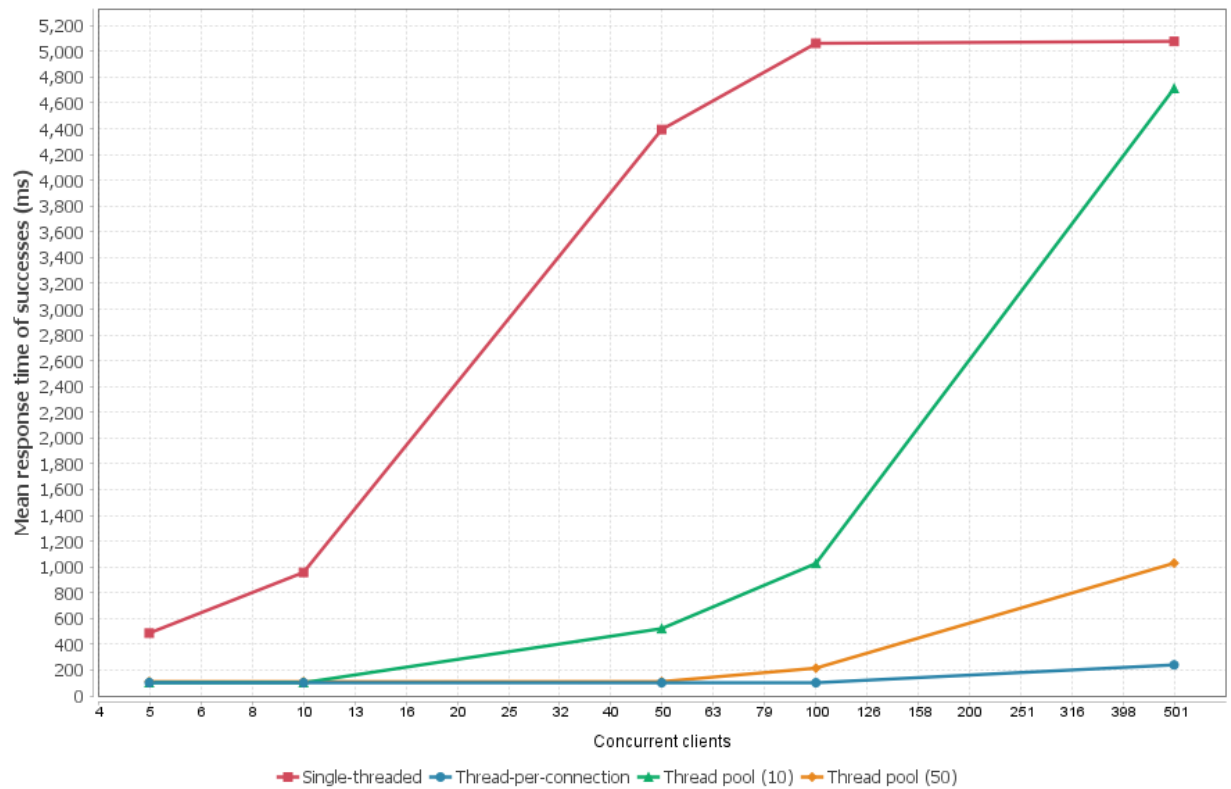
A2: Within the executor service there is no implementation that does a true thread per connection. The closest is the CachedThreadPool executor which will create a thread if a currently created thread is not available, but if a thread becomes idle for more than 60 seconds the executor will kill this thread, meaning the thread is not guaranteed to live on the entire application and a thread is not guaranteed to be made upon every connection. This is why we choose to go with the original implementation of creating a new thread upon every connection given the requirement of creating a new thread upon new connections.

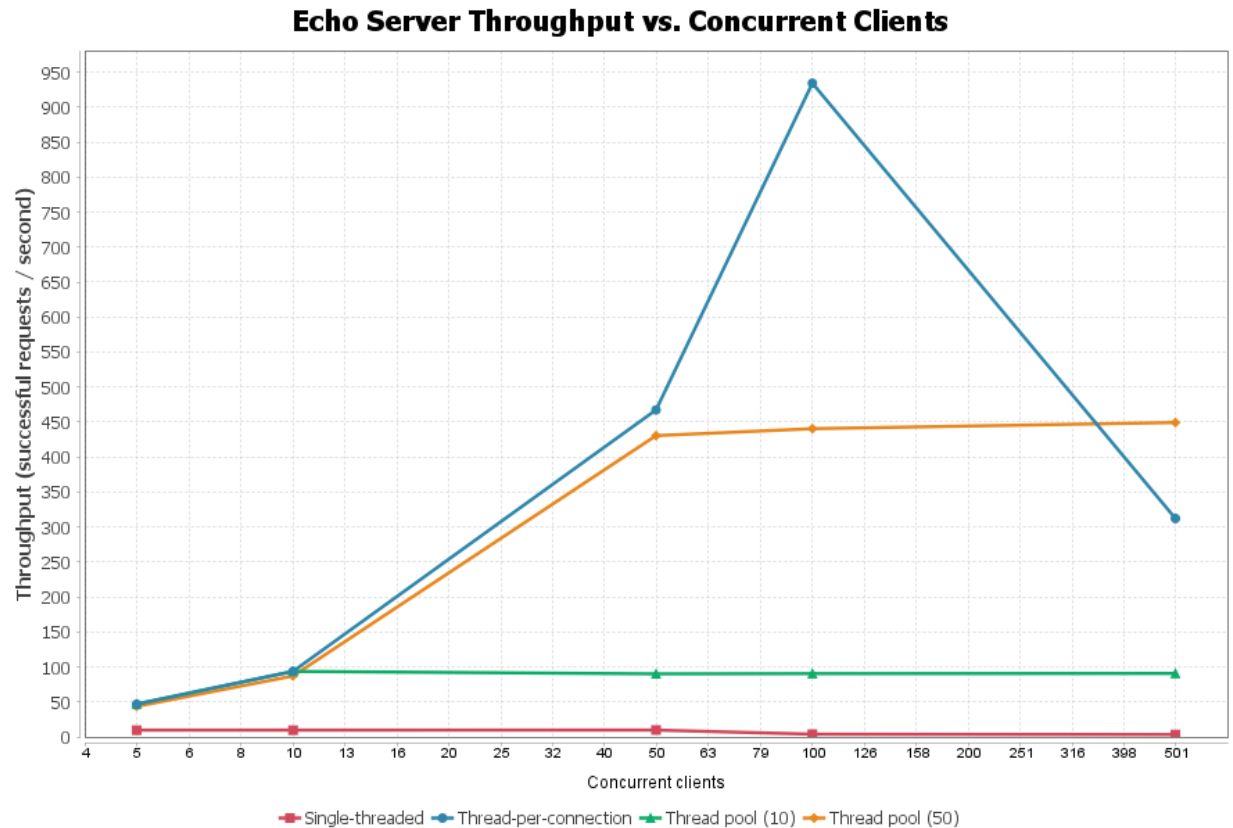
Q3: Which strategy is the best for 5/50/100 or 500? Must provide evidence

Echo Server Error Rate vs. Concurrent Clients



Echo Server Response Time vs. Concurrent Clients





A3: Which strategy is the best for the given client counts depend entirely on client count, we will use the figures above to show this.

One thing to point out is that only using a single thread is clearly a loser amongst all figures. Using a single thread we can see we have the lowest throughput, highest response time, and start encountering errors earlier than any concurrent pattern.

At 5 and 10 clients every concurrent strategy performs identically. This is because the offered load never exceeds the number of available workers so no request ever is waiting for available resources. Yet if we were to choose a winner I would choose thread pool 10 given it only allocates 10 threads, meaning it uses less resources.

At 50 clients the strategies begin to separate. Figure 3 shows that threadpool 50 and thread per connection are still increasing its throughput linearly, with thread per connection starting to take a slight advantage. On the other hand the thread pool of 10s throughput flatlines at around 100 successful request per second indicating an overhead bottleneck and a queueing of clients waiting for a thread to become available. This bottleneck can also be observed in figure 2 which shows the response time for threadpool grows larger as

its queue grows larger. The winner at 50 clients would be thread pool 50 given it only binds 50 threads.

At 100 clients thread per connection is clearly the fastest winning in both throughput and response time. Figure 3 shows that clients thread per connection can has roughly double the throughput than thread pool 50 and also has a faster response time than thread pool 50 due to the therapods queue starting to grow larger. At this client level thread per connection also has yet to encounter an error, indicating at a 100 client load, the system can support it.

At 500 clients the story flips on the thread per connection solution. We see a total collapse of the thread connection approach showing in figure 1 as soon as we increase past 100 clients errors start linearly increase. Figure 3 also shows a similar story where for the thread per connection solution throughput is linearly decreasing until it eventually has a lower throughput than the thread pool of 50 solution. This catastrophic failure for the thread per connection solution shows the real computing cost of making a new thread on connection instead of simply allocating a fixed count of threads to handle accepting of connections. At 500 clients per connection the best choice would be the pool of 50 supporting a better throughput than any other solution in our experiment while also having a reasonable response time and no errors.

Overall the pattern we observed in this experiment is that unbounded strategies are faster all the way up to their failure, and when they fail, they fail badly. The bounded solutions may provide a lower throughput and higher response time but guarantee availability of they system with no catastrophic failures related to load. The clearest example of this within the experiment would be the 500 client example. If we had to choose a strategy without knowing the load ahead of time we would choose a bounded thread strategy (I.E 10 thread pool or 50 thread pool) where the number of threads is some n number of threads deemed safe for the system until we are able to stress test the load ability and choose the most optimal number of threads for the applications use case.